QA Test Prompts

SmartAg Ops Advisor - copy-paste prompts for Claude Code / Copilot or manual runs. Companion to QA_Testing_Prompt.pdf (generic end-to-end prompt).

#	Prompt	When
P1	Backend acceptance tests AT-01..AT-18	Every merge; before freeze
P2	Regression: remove user	After each backend deploy
P3	Regression: Reports page	After each backend deploy
P4	Deployment check (Render)	Before any UI retest
P5	Decision flow result states	Before demo; 07:00 dry run
P6	Data honesty	Before demo

How to use: paste one prompt per session. Fill the bracketed values. Save outputs in `./qa-tests/`. Log every failure with the bug template on the last page.

P1 - Backend acceptance tests AT-01 to AT-18

Owner: QA + Dev. Run before every merge and before the 00:30 freeze.

```
Repo: agri-project/ (backend = FastAPI, tests in backend/tests/).
STRICT RULE: Do not modify any application code (frontend, backend,
config, DB schema, package files). Only create new files under ./qa-tests/.
If you find a bug, document it with steps, expected vs actual, and the
likely file:line. Never delete or edit real users or data on the shared
Neon database; create throwaway test accounts named qa_<date>_<n> and
remove only those.

1. cd agri-project/backend, activate venv, run: python -m pytest -v
2. Map every test to its acceptance ID (AT-01 .. AT-18, AT-09b) from
   CLAUDE.md section 5. Flag any AT ID that has no test.
3. For each AT, re-check the expected numbers by hand from the brief
   (e.g. AT-14: A 11,500, B 10,700, C 11,800 -> C first by 300;
   AT-17: left 4 and 10 per crate, cover 116 and 140; AT-18: TOO_CLOSE 50<100).
4. Confirm money maths uses Decimal and rounding happens only for display.
Output: qa-tests/AT_REPORT.md table: AT ID | test name | PASS/FAIL | notes.
Overall X passed / Y failed.
```

P2 - Regression: Admin > Remove user

Bug fixed 25 Oct: 'Could not remove this user.' (500 from audit log reading a deleted row).

```
App: Admin page (/admin), endpoint DELETE /api/auth/users/{id}.
STRICT RULE: Do not modify any application code (frontend, backend,
config, DB schema, package files). Only create new files under ./qa-tests/.
If you find a bug, document it with steps, expected vs actual, and the
likely file:line. Never delete or edit real users or data on the shared
Neon database; create throwaway test accounts named qa_<date>_<n> and
remove only those.

Sign in as admin. Create throwaway accounts first, then test:
 a) Farmer with crops, a booking (with location points), a chat
    conversation, notifications and an uploaded document -> removed,
    HTTP 204, user disappears from the list, no error banner.
 b) Truck operator who owns a truck -> user removed; truck still exists,
    operator unlinked and status 'inactive'; not bookable.
 c) Market manager who published a price -> price kept, created_by empty.
 d) Admin tries to remove own account -> "You cannot remove your own account".
 e) Remove the last remaining admin -> "At least one admin account must remain".
 f) Remove an id that no longer exists (double click / two tabs) -> "User not found".
 g) Removed user's open session -> next request signs them out to /login.
 h) Non-admin calls DELETE directly -> 403.
For each case record the HTTP status, the exact message shown in the UI,
and DevTools console/network errors. The generic text
"Could not remove this user." must never appear for these cases.
```

P3 - Regression: Reports page

Bug fixed 25 Oct: 'Report unavailable' (500 when government prices were dicts).

```
App: /reports, endpoint GET /api/reports/summary.
STRICT RULE: Do not modify any application code (frontend, backend,
config, DB schema, package files). Only create new files under ./qa-tests/.
If you find a bug, document it with steps, expected vs actual, and the
likely file:line. Never delete or edit real users or data on the shared
Neon database; create throwaway test accounts named qa_<date>_<n> and
remove only those.

For each role (farmer, truck_operator, market_manager, admin):
 1. Open /reports. Page must load (no 'Report unavailable').
 2. Check sections render: total stock, estimated net, actual net,
    transport cost, stock by crop, financial summary, booking
    performance, market coverage, recent bookings.
 3. Cross-check numbers with the API JSON (DevTools > Network):
    stock.total_kg, bookings.total/active/delivered, financials.*.
 4. markets.stale_prices must equal the count of prices with
    is_stale = true in markets.prices (it used to be stuck at 0).
 5. Brand-new account with no crops/bookings -> empty states shown,
    not an error.
 6. Stop the backend (or go offline) -> error card with 'Try again';
    restart -> 'Try again' loads the report.
 7. Print report button opens print preview with readable layout.
Output: qa-tests/REPORTS_REPORT.md, one row per role per check.
```

P4 - Deployment check (Render)

The frontend calls https://smartag-ops-backend.onrender.com, not local code.

```
STRICT RULE: Do not modify any application code (frontend, backend,
config, DB schema, package files). Only create new files under ./qa-tests/.
If you find a bug, document it with steps, expected vs actual, and the
likely file:line. Never delete or edit real users or data on the shared
Neon database; create throwaway test accounts named qa_<date>_<n> and
remove only those.

1. Note the commit on origin/main (git log origin/main -1) and the
   commit Render says it deployed (Render dashboard > Events).
   They must match before any UI retest of P2/P3.
2. GET /api/health -> 200. GET /openapi.json lists
   /api/auth/users/{user_id} and /api/reports/summary.
3. Cold start: time the first request after 15 min idle (free plan
   sleeps). Record it; a sleeping backend can look like an app bug.
4. CORS preflight for DELETE from the frontend origin returns
   access-control-allow-origin for that origin.
5. Confirm frontend VITE_API_URL points at the backend you expect.
Report: deployed commit, matches main (Y/N), health, cold-start seconds.
```

P5 - Decision flow result states (demo path)

Must-haves 1-3 from CLAUDE.md. Run against the guest comparison screen.

```
Spec: CLAUDE.md sections 4-5 (PRD v4.1). Endpoints expected:
GET /api/options, POST /api/compare, GET /api/health.
STRICT RULE: Do not modify any application code (frontend, backend,
config, DB schema, package files). Only create new files under ./qa-tests/.
If you find a bug, document it with steps, expected vs actual, and the
likely file:line. Never delete or edit real users or data on the shared
Neon database; create throwaway test accounts named qa_<date>_<n> and
remove only those.

0. First confirm POST /api/compare and GET /api/options exist and are
   reachable without login. If missing, report as BLOCKER and stop.
1. Drive the UI to each state and screenshot it:
   RANKED, TOO_CLOSE, NEEDS_INPUT, INSUFFICIENT_MARKETS, NO_FRESH_DATA,
   SOURCE_ERROR, EMPTY. Message text must match the brief exactly.
2. Only RANKED shows a winner; winner_market_id is null otherwise.
3. Blank trip cost != zero: zero needs the 'no cost' tick (NEEDS_INPUT).
4. Field errors with no calculation for: quantity <= 0, negative cost,
   capacity 0.
5. Screen order: decision box -> harvest cover panel -> cost inputs ->
   market cards -> excluded markets with reasons -> assumptions strip.
6. When no market covers costs, the message is the exact brief text;
   the app never says "don't harvest".
7. Mobile viewport 375px: no horizontal scroll, all states readable.
```

P6 - Data honesty

Every price must be traceable; nothing is called 'live'.

```
STRICT RULE: Do not modify any application code (frontend, backend,
config, DB schema, package files). Only create new files under ./qa-tests/.
If you find a bug, document it with steps, expected vs actual, and the
likely file:line. Never delete or edit real users or data on the shared
Neon database; create throwaway test accounts named qa_<date>_<n> and
remove only those.

1. On every screen of the demo path, each price shows source, report
   date, unit and market type.
2. Sample mode: badge "Sample data as of 25 Oct 2026" on every screen
   (AT-15). Snapshot mode: "Reported 24 Oct 2026, source: AGMARKNET via
   data.gov.in".
3. Search UI text and API responses for the word "live" near prices -> none.
4. One comparison never mixes sample and snapshot rows.
5. Excluded markets show a plain reason: stale, unit unclear,
   different market type, different report date, price invalid.
6. No API key, DATABASE_URL or token appears in responses, page
   source, console logs or committed files (git grep).
```

Bug report template

```
ID:            BUG-<nn>
Title:         <one line>
Severity:      Blocker / Major / Minor
Page / API:    e.g. /admin  -  DELETE /api/auth/users/12
Role / user:   e.g. admin (qa_1025_1)
Steps:         1. ...  2. ...  3. ...
Expected:      ...
Actual:        ... (exact UI message)
Evidence:      screenshot path, HTTP status, response body, console error
Likely cause:  file:line (do not fix)
Build:         git commit + backend URL tested
```

Exit criteria for the demo

All P1 tests pass; P2 and P3 pass on the deployed backend; P5 shows every state with exact messages; P6 finds no unlabelled price and no secret.